

Capstone Project Proposal

Grade: B

Status: Pass (automatic by LLM)

Feedback

Capstone Project Proposal Evaluation

Score

85/100 -- Good

Uncapped score: 97/100

Applied cap: No plan to deploy as a Databricks App (maximum final score of 85)

Rubric Breakdown

Category	Score	Maximum	Feedback
Project definition and data sources	15	15	Clear problem, users, and workflow. Multiple concrete sources and APIs identified; formats and scales are specified (e.g., 2.4M TSB rows, investigations row count).
Spark data pipeline	10	10	Well-specified Bronze/Silver/Gold pipeline with Auto Loader, Structured Streaming, normalization, dedupe, ai_extract, chunking, idempotent upserts; coherent downstream connections.
Third-party API integration	10	10	Specific APIs (NHTSA Recalls, vPIC) with endpoints, polling frequency (60s), caching strategy, and rate-limit considerations; keyless auth noted.
Lakebase data model	10	10	Lakebase as OLTP is clearly defined; schema enumerated with relationships implied; read/write access patterns from tools and agent described.
Action-taking AI agent	15	15	Purpose and tools are explicit. Retrieval via AI Search; meaningful write tools (open_defect_signal, assign_work_order, launch_service_campaign) with audit and approval gate; execution context/governance clear.
Analytics pipeline	10	10	Lakebase CDF to Delta plus DLT fact tables; events captured and useful metrics/dashboards well described.
Frontend and deployment	7	10	Frontend workflows and identity flows are clear and connected to data/agent. However, deployment is planned on Render, not as a Databricks App, which is required for full credit.
Big Data characteristics	10	10	Volume (>1M rows) and Variety (unstructured narratives) are credibly justified. Velocity is explicitly "partial" and does not meet <1-minute threshold.
Architecture diagram	10	10	Two readable PNG diagrams included; components and flows align with the write-up.
Total	85	100	

Requirements Check

- Spark data pipeline: Meets
- Third-party API: Meets
- Lakebase data model: Meets
- Agent retrieval capability: Meets
- Agent write action: Meets
- CDF or Delta Live Tables analytics pipeline: Meets
- Frontend: Meets
- Databricks App deployment plan: Does Not Meet
- Big Data Volume: Meets
- Big Data Velocity: Does Not Meet
- Big Data Variety: Meets

- At least two qualifying Vs: Meets
- Written data-integration plan: Meets
- PNG or JPEG architecture diagram: Meets

Big Data Assessment

- Qualifying Vs:
- Volume: Meets. Evidence includes 2.4M TSB rows plus full complaints/recalls/investigations; ingestion and schema stability verified.
- Variety: Meets. Owner-written free-text narratives and other prose are processed (chunking, embeddings, AI Search).
- Non-qualifying V:
- Velocity: Does not meet. Proposed detection path is 60-second API polling plus a 30-second micro-batch, which exceeds the <1-minute requirement.

Strengths

- Exceptionally clear problem framing, target users, and measurable validation using ODI investigation dates.
- Thorough, feasible Spark/Lakehouse design: streaming bronze->silver->gold, normalization, expectations/quarantine, AI Search, Feature Store, and serverless/Photon choices.
- Action-taking agent with governed UC Functions, meaningful write tools, human approval gate, and full audit/traceability via MLflow 3.
- Strong analytics plan using Lakebase CDF and DLT, with operationally relevant metrics (e.g., human override rate).

Required Revisions

- Provide a concrete plan to deploy the frontend as a Databricks App (required). Clarify:
 - How the current FastAPI/Jinja UI will be packaged and hosted as a Databricks App.
 - Identity/authorization in-App (OBO user flow) and how Lakebase credentials will be handled (Apps rotate automatically).
 - How the App will invoke Model Serving and UC Function tools, and how ABAC continues to enforce row/column security.
 - Whether the public surface remains external or also ships as an App; if external remains, state that the Databricks App will support the core operator workflow at minimum.
- If you want Velocity credit, commit to and justify <60s end-to-end latency, for example:
 - Reduce polling interval (e.g., 15s with backoff) and keep micro-batch at ?15s, or switch to a pattern that achieves sub-minute freshness while honoring API limits.
 - Provide a brief calculation of expected worst-case vs. average latency.
- Minor clarifications to strengthen completeness:
 - Add an expected row count for the complaints corpus and an example of partitioning strategy.
 - Note CDF retention settings for Lakebase-backed analytics tables and any monitoring around CDF lag.

Overall Feedback

This is an excellent, production-grade proposal: the data pipeline, operational

model, agent tooling, and analytics are specific, feasible, and well-governed. The only substantive blocker against a top score is the lack of a Databricks App deployment plan for the frontend, which is an explicit assignment requirement. Add an App deployment path for the operator console (and optionally keep the external public surface) and, if desired, tighten the latency plan to qualify for Velocity. With those adjustments, this would score in the excellent band.